

Cell Differentiation Task

Sreedev M

August 24, 2026

Contents

1. Idea	3
2. Design	4
2.1. Vanilla NCA	5
2.2. Non Local NCA	5
2.2.1. v1 (Discarded)	6
2.2.2. v2	6
2.3. Initialization	7
3. Experiments	8
3.1. Convergence Time Comparison	8
3.1.1. Results	9
3.1.2. Analysis	9
Bibliography	10
Index of Figures	11

1. Idea

The idea was to devise a toy problem of the sort where each individual cell does not have a pre-defined target. The property trained for should be measured at the grid level, which would subsequently mean that loss would also be applied at grid level.

Such a problem can bring out the difference between a Moore-neighbourhood based NCA and a NCA with non-local connections better than problems with cell level targets. Using such a problem, we can test different versions of the non-local NCA and treat the advantage over vanilla NCA as a “Hello World” check.

The problem is defined as follows:

1. Each grid starts out with each cell in a parent state.
2. A parent state can mutate into a daughter state A, or daughter state B, or stay as it is.
3. The goal is to learn a set of update rules, so that the NCA settles into a configuration, where the ratio of daughter A to daughter B is a predefined target, and the number of cells in the parent state is as minimum as possible.

It is entirely possible for the vanilla NCA to solve this task. The advantage discussed earlier would be in the form of convergence time, since immediate neighbour-neighbour diffusion would take a longer time than non-local version.

- Since if a cell has a predefined target, it can reach there without little consideration given towards the state of the grid.

- where information can travel faster; if it is a small-world map, information can travel within $\log(N)$ steps where N is the grid size.

2. Design

The vanilla NCA architecture is pretty much the same as that of the one discussed in A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin [1]. Main differences are as follows:

- We will have no hidden channels for the first few experiments. We will observe the role hidden channels play by varying them after initial experiments are performed. Similarly, the firing rate of cells is kept at 1, and will be varied later.
- The Sample Pool structure is kept as is from [1]. The worst performing grid is replaced with a grid at initial state after every epoch

Some task specific design choices:

1. The main channel is responsible for the **fate** of the cell.

$$\text{fate}(f) = \begin{cases} \text{parent} , & -0.95 \leq f \leq 0.95 \\ \text{daughter A} , & -1 \leq f < -0.95 \\ \text{daughter B} , & 0.95 < f \leq 1 \end{cases} \quad (1)$$

The update output from the network is capped at $\text{mod}(0.05)$ and the cell fate is capped at $\text{mod}(1)$.

2. Loss functions

1. **Commitment Loss:** a double wellled potential with two minima, one at -1 and the other at 1 , and a maximum at 0 . This forces the cells to move away from the parent state towards the two daughter states.

$$\mathcal{L}_{\text{commitment}} = (1 - \text{fate}^2)^2 \quad (2)$$

2. **Ratio Loss:** quantifies the ratio of the grid from the target ratio. This is done with the help of a helper function called strength.

$$\begin{aligned} \text{strength}(A) &= \sum_{\text{grid}} \text{ReLU}(\text{fate}) \\ \text{strength}(B) &= \sum_{\text{grid}} \text{ReLU}(-\text{fate}) \end{aligned} \quad (3)$$

Then ratio of B is defined as

$$\text{ratio}_B = \frac{\text{strength}(B)}{\text{strength}(A) + \text{strength}(B)} \quad (4)$$

The ratio loss is then

$$\mathcal{L}_{\text{ratio}} = E_{\text{batch}} [(\text{ratio}_B - \text{target ratio})^2] \quad (5)$$

• with Moore neighbours

• strength quantifies the commitment towards a daughter

• The ratio loss is computed across the batch used during training epoch.

3. **Variance Loss** : If the batch size gets large enough, the network can push each individual grid within the batch towards all daughter cell A or all daughter cell B and try to achieve the ratio at the batch level instead of the grid level. In order to mitigate this we try to minimize the variance between grids.

$$\mathcal{L}_{\text{variance}} = E_{\text{batch}} \left[(\text{ratio}_B - E_{\text{batch}}[\text{ratio}_B])^2 \right] \quad (6)$$

The total loss is

$$\mathcal{L} = \mathcal{W}_{\text{commitment}} * \mathcal{L}_{\text{commitment}} + \mathcal{W}_{\text{ratio}} * \mathcal{L}_{\text{ratio}} + \mathcal{W}_{\text{variance}} * \mathcal{L}_{\text{variance}} \quad (7)$$

where $\mathcal{W}$ s are the weights to the terms.

2.1. Vanilla NCA

The pipeline is given below:

1. An identity filter, Sobel X filter, and Sobel Y filter are convolved over the grid ($C \rightarrow 3C$)
2. A CNN with 3 Layers act on this grid with $3C$ channels:

• C is the number of channels

Layer 1	2D Convolution , 32 Channels, ReLU, Width 1	$3C \rightarrow 32$
Layer 2	2D Convolution , 32 Channels , ReLU, Width 1	$32 \rightarrow 32$
Layer 3	2D Convolution , C Channels , No Activation, Width 1	$32 \rightarrow C$

3. The output of Layer 3 is the update given to the current grid.
4. The grid is evolved for some steps and the loss is computed. The gradients are updated via BPTT.
5. The worst performing grid is removed and replaced with a grid in the initial state
6. Steps 1 to 5 are repeated for some number of times.

• who has the highest variance in the batch

2.2. Non Local NCA

Non locality is established by randomly selecting a fixed percentage of cells in the grid to have one long range partner. The partner is also randomly chosen, and their distance must be greater than some fixed Manhattan Distance . A wiring bank is made by repeating the long range wiring process for N_{wirings} times. When a batch is selected from the pool during train, each sample within the batch is chosen to be a unique wiring from the wiring bank.

• here it is 10

• since it is random, each wiring process generates a different graph

• this wirings associated with a batch change in each epoch

This ensures that the network learns rules which generalize to all kinds of wirings..

2.2.1. v1 (Discarded)

1. An identity filter, Sobel X filter, and Sobel Y filter are convolved over the grid ($C \rightarrow 3C$)
2. A CNN (termed Local CNN) with 3 Layers act on this grid with $3C$ channels:

Layer 1	2D Convolution , 32 Channels, ReLU, Width 1	$3C \rightarrow 32$
Layer 2	2D Convolution , 32 Channels , ReLU, Width 1	$32 \rightarrow 32$
Layer 3	2D Convolution , C Channels , No Activation, Width 1	$32 \rightarrow C$

3. For each wiring, the parings are determined. The channel values of the partners are collected and concatenated. A CNN (termed Non Local CNN) with 3 Layers act on this grid with $3C$ channels:

Layer 1	2D Convolution , 32 Channels, ReLU, Width 1	$1 \rightarrow 32$
Layer 2	2D Convolution , 32 Channels , ReLU, Width 1	$32 \rightarrow 32$
Layer 3	2D Convolution , C Channels , No Activation, Width 1	$32 \rightarrow C$

4. The Local CNN and the Non Local CNN outputs their own separate update grid. These are then independently added to the current grid and evolved.
5. The rest of the training procedure proceeds similar to the vanilla NCA.

This version of the Non Local NCA is discarded now and the reasons are discussed later.

2.2.2. v2

1. An identity filter, Sobel X filter, and Sobel Y filter are convolved over the grid ($C \rightarrow 3C$)
2. For each wiring, the parings are determined. Three additional channels are created which will contain the partnter's identity,

• which is a function of fraction of cells with wiring and the Manhattan Distance

• C is the number of channels

• If the cell deoesnt have a partner, zero vectors are concatenated, so the lenght of the vector of NC

Sobel X and Sobel Y values respectively. If a cell doesn't have a partner the values are kept at 0. (3C → 6C)

◦ throughout the process

3. A CNN with 3 Layers act on this grid with 3C channels:

Layer 1	2D Convolution , 32 Channels, ReLU, Width 1	6C → 32
Layer 2	2D Convolution , 32 Channels , ReLU, Width 1	32 → 32
Layer 3	2D Convolution , C Channels , No Activation, Width 1	32 → C

4. The output of Layer 3 is the update given to the current grid.
5. The rest of the training procedure proceeds similar to the vanilla NCA.

2.3. Initialization

Each cell in the grid is initialized with a small random noise at 0. The weights of the network are also randomly chosen. The probability of a cell getting a long range partner is determined by a hyperparameter. A small fraction of cells are then converted to daughter A/ daughter B cells with random chance so that there is variability and symmetry-breaking in the initial state.

■ another hyperparameter

3. Experiments

3.1. Convergence Time Comparison

1. N_R number of ratios are randomly chosen between the range 0.35 and 0.75. Each N_R is associated with N_s number of seeds. The seeds are same across ratios.
2. The seed determines the model weights, the initial grid etc. so that both the vanilla NCA and Non Local NCA having almost everything same except the wiring topology.
3. A vanilla NCA model and a Non Local NCA Model is trained for every seed associated with every ratio.
4. N_{eval} number of fresh initial grids are then made and it is evolved with the learned rules till it reaches within tol. The maximum number of steps it can take is set as MAX_STEPS and the models that fail to converge before this are discarded.
5. The number of steps taken for convergence is noted down for both vanilla NCA and Non Local NCA. An average is taken across the evaluation grids and this number can be denoted as $t_{s_i}^{r_j}$

0 to 1 was not chosen because it was getting harder to train

for 1000 epochs

tol is set here as within 0.05 band of target ratio and less than 5% parent cells

Work has to be done to bring this to a minimum.

satisfying the tol condition

$$6. \quad \frac{1}{N_s + N_r} \sum_{s_i} \sum_{r_j} t_{s_i}^{r_j} \quad (8)$$

is the mean convergence time. This is shown as the point in the plot.

$$7. \quad \frac{\sigma_{\{s_i\} \cup \{r_j\}}}{\sqrt{N_s * N_r}} \quad (9)$$

is the mixed standard deviation of the readings and this is plotted as the error-bar

8. $(\sigma_{\{s_i\}})$ is the standard deviation of the readings across seed axis and this is plotted as a band.
9. Steps 4 to 8 are repeated again for grids of different sizes in N_{test}

3.1.1. Results

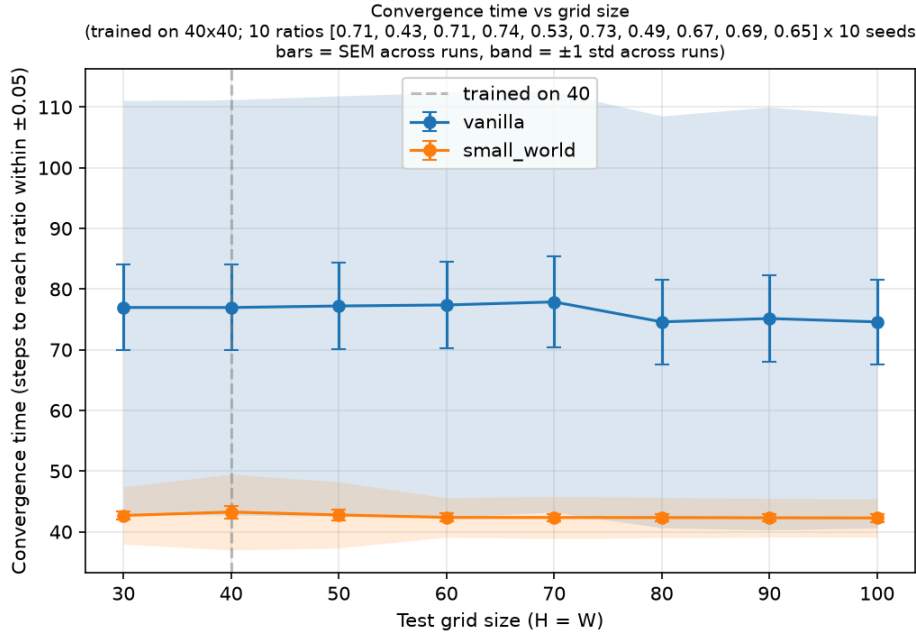


Figure 1: Convergence Time vs Grid Size for vanilla NCA and Non Local NCA with $p = 0.3$. The mean convergence time are the points, $\frac{\sigma}{\sqrt{N}}$ of the whole dataset is the error-bar, and σ across seeds is the band.

3.1.2. Analysis

- From Figure 1 it is evident that Non Local NCA has a clear advantage in convergence time. Plus the convergence time across seeds are stable for Non Local NCA compared to vanilla NCA which are all over the place. Does this mean that the Non-Local NCA converged to a more general rule across seeds, wirings and initializations that is not affected by perturbation?
- Since the channels 2-5 are always set to 0 for the vanilla NCA, a reason for the slower performance of vanilla NCA might be due to effectively working with a lower parameter count. There is a need to perform the experiment with the effective parameters of vanilla NCA either matching or exceeding that of its non local counterpart to strengthen the claim for non local connections.

• denoted as small-world

• Parameters for Figure 1

N_r	10
N_s	10
N_{eval}	10

• notice the small band for non local NCA compared to a very wide band for vanilla

Bibliography

- [1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, “Growing Neural Cellular Automata,” *Distill*, 2020, doi: 10.23915/distill.00023[◦].

Index of Figures

Figure 1 Convergence Time vs Grid Size for vanilla NCA and Non Local NCA with $p = 0.3$. The mean convergence time are the points, $\frac{\sigma}{\sqrt{N}}$ of the whole dataset is the error-bar, and σ across seeds is the band. 9

◊ denoted as small-world

■ Parameters for Figure 1

N_r	10
N_s	10
N_{eval}	10